Lab 11 — Task 1: Key AI & NLP Concepts

Name: Syed Tahir Ali **Roll No:** SU92-BSAIM-F24-103 **Subject:** Programming for AI Lab **Section:** BSAI-4B

---

1. LangChain

LangChain is an open-source Python framework that helps developers build applications powered by Large Language Models. Instead of writing low-level integration code from scratch, LangChain provides ready-made components — chains, retrievers, memory, and agents — that can be combined into a complete AI pipeline quickly and cleanly.

Think of LangChain as the "glue" of an AI application. It handles document retrieval, prompt formatting, LLM calls, and response parsing automatically, so the developer can focus on the actual application logic rather than plumbing.

Key capabilities:

- Connects LLMs to external databases, APIs, and document stores
- Supports conversation memory so the chatbot remembers previous messages
- Enables agents that can autonomously use tools like web search or a calculator
- Works with many LLM providers: OpenAI, Groq, Gemini, Ollama, and more

Real-Life Example: The SuperiorInsight chatbot built in this lab is a direct example. LangChain connects a FAISS vector database containing Superior University's scraped data to the Groq LLM. When a student asks "What are the credit hours for BSCS?", LangChain retrieves the relevant document chunks and passes them to the LLM, which generates a precise and grounded answer instead of guessing.

---

2. LLMs (Large Language Models)

Large Language Models are deep learning models trained on billions of words from books, websites, code, and articles to learn how human language works. They are built on the Transformer architecture and, after training across hundreds of billions of parameters, they can understand context, answer questions, summarize documents, translate languages, and generate code — all from a single prompt.

What makes LLMs special compared to older AI systems is that they are completely general-purpose. A traditional NLP model was trained for one specific task like sentiment analysis. An LLM like GPT-4 or Gemini handles hundreds of different tasks without any extra training. The

generation process is simple in concept: given your input, the model predicts the most likely next word, then the next, continuing until the full response is complete. At billions of parameters, this produces remarkably intelligent output.

Key characteristics:

- No task-specific retraining needed — one model, many use cases
- Understand context across long conversations
- Examples: GPT-4 (OpenAI), Gemini (Google), LLaMA 3 (Meta), Claude (Anthropic), Mistral

Real-Life Example: When you ask ChatGPT to explain a concept, debug your code, or draft a formal email — that is an LLM in action. GitHub Copilot is another example: it watches what you are typing in your code editor and suggests the next lines of code, having learned from millions of public GitHub repositories.

---

3. RAG (Retrieval-Augmented Generation)

RAG is an AI architecture that solves one of the biggest weaknesses of LLMs: they only know what they were trained on. If you ask a pure LLM about your university's current fee structure or a policy published last month, it will either hallucinate an answer or admit it does not know.

RAG fixes this by giving the LLM access to a real knowledge base at the time of the query. When a user asks a question, the system first searches a database of documents for the most relevant chunks, then feeds those chunks alongside the question into the LLM as context. The LLM generates its answer based on that retrieved information — not from guesswork.

How it works step by step:

1. User submits a question
2. Question is converted into a vector embedding
3. Similarity search runs against the Vector DB
4. Most relevant document chunks are retrieved
5. Those chunks plus the question are sent to the LLM
6. LLM generates a grounded answer from the real documents

Real-Life Example: SuperiorInsight is a RAG system. The scraped Superior University data and PDFs are stored in FAISS. When a student asks about a program, RAG pulls the actual university documents and the LLM answers from that real content — not from imagination. Microsoft Copilot in Teams and Word works the same way, searching your company's own files to answer your questions.

---

4. FAISS (Facebook AI Similarity Search)

FAISS is an open-source library built by Meta (Facebook AI Research) designed to search through massive collections of vectors and find the most similar ones extremely fast. Traditional databases are great at exact matches — "find the row where ID equals 5." But they cannot efficiently answer semantic questions like "find the 4 documents most similar in meaning to this sentence." FAISS was built exactly for that problem.

It uses smart mathematical indexing techniques — such as Hierarchical Navigable Small World graphs and Inverted File Indexes — to search millions of vectors in milliseconds. It supports both exact and approximate nearest-neighbor search and runs on both CPU and GPU.

How it measures similarity:

- Cosine Similarity — measures the angle between two vectors, most common for text
- L2 Euclidean Distance — measures the straight-line distance between vectors in space

Real-Life Example: In SuperiorInsight, when you ask "What is the fee structure?", your question is converted into a vector and FAISS searches thousands of stored document vectors, returning the 4 most similar chunks almost instantly. On a larger scale, Instagram uses FAISS internally to find visually similar images across billions of photos and recommend posts you might like.

---

5. Vector

In AI and machine learning, a vector is a fixed-size list of floating-point numbers that represents the semantic meaning of a piece of text, an image, or any other data. When a sentence passes through an embedding model, it is converted into a vector of maybe 384 or 768 numbers. The key insight is that the model places similar meanings close together in this numerical space — so sentences with the same meaning end up near each other even if the words are completely different.

Example:

- "university admission fee" → `[0.12, -0.45, 0.89, 0.33, ...]` — 384 numbers
- "tuition cost for students" → `[0.11, -0.43, 0.91, 0.31, ...]` — very close, same meaning
- "cricket match yesterday" → `[-0.82, 0.67, -0.23, 0.55, ...]` — far away, unrelated topic

This is fundamentally different from keyword search. A keyword search misses the connection between "admission fee" and "tuition cost" because the words differ. Vectors capture meaning, not words — enabling true semantic search.

Real-Life Example: Google Search has moved beyond keyword matching to vector-based semantic search. When you type "cheap flights to Lahore", Google also returns results for "affordable air tickets to Pakistan" because the vector representations of those queries are mathematically close. The search engine understands what you mean, not just what you typed.

---

6. Vector DB (Vector Database)

A Vector Database is a specialized database built to store, index, and query vector embeddings at scale. Regular databases like MySQL are designed for structured data and exact queries. A Vector DB is designed for a completely different purpose — finding the most semantically similar items to a given input, efficiently, across millions or billions of stored vectors.

Under the hood, Vector DBs use approximate nearest-neighbor algorithms so they never have to compare every single vector in the collection, which would be impossibly slow. This makes them practical for real-time AI applications.

Popular Vector Databases:

- FAISS — fast local library, used in this lab for RAG
- Pinecone — cloud-managed, built for production-scale apps
- ChromaDB — lightweight, great for development and small projects
- Weaviate — open-source with hybrid keyword and vector search
- Qdrant — high-performance open-source for production

Real-Life Example: Spotify uses a Vector DB to power its music recommendations. Every song is converted into a vector based on its audio features — tempo, energy, pitch, and mood. When a track finishes, Spotify queries the Vector DB for the nearest song vectors and recommends them. That is why Spotify recommendations feel like they genuinely understand your taste rather than just picking songs from the same genre.

---

7. Generative AI (GenAI)

Generative AI is a broad category of AI systems that can create new, original content from scratch — text, images, audio, video, code, or 3D models. Unlike traditional AI that classifies or predicts from existing data, a generative model produces something that did not exist before. These

systems learn the statistical patterns and styles present in their training data, then use those learned patterns to generate new, authentic-feeling output from a prompt.

Types of Generative AI:

- LLMs generate text and code — examples include GPT-4, Gemini, and Claude
- Diffusion models generate images — examples include DALL·E 3 and Midjourney
- Voice models generate speech and music — examples include ElevenLabs and Suno AI
- Video models generate video clips — examples include Sora by OpenAI and Runway ML
- Code models generate programming code — examples include GitHub Copilot and Cursor

Real-Life Example: Adobe Firefly lets a graphic designer type "a futuristic city skyline at sunset with neon lights and flying cars" and generates a photorealistic image in seconds. That image has never existed before — it was created entirely from patterns the model learned during training. Similarly, a student can ask ChatGPT to write a Python function, explain a concept, or draft an email — all generated on the spot, not copied from anywhere.

---

8. GANs (Generative Adversarial Networks)

GANs are a deep learning architecture where two neural networks are trained simultaneously in direct competition with each other. The first network, called the Generator, takes random noise as input and tries to produce fake data — images, audio, or text — realistic enough to be mistaken for real. The second network, called the Discriminator, is shown both real data and the Generator's fakes and tries to correctly identify which is which.

The two networks improve through a continuous feedback loop. When the Discriminator catches a fake, the Generator learns from that failure and produces better fakes next time. When the Generator fools the Discriminator, the Discriminator learns to become sharper. After thousands of training iterations, the Generator can produce output so realistic that even humans cannot distinguish it from real data.

Training process:

1. Generator receives random noise and produces a fake output
2. Discriminator sees both real samples and the fake output
3. Discriminator tries to classify each as real or fake
4. Both networks receive feedback and update their weights
5. This cycle repeats thousands of times until the Generator becomes highly convincing

Real-Life Example: The website thispersondoesnotexist.com generates a completely new photorealistic human face every time you refresh the page. None of these people exist — every

face is created by NVIDIA's StyleGAN, a GAN architecture. On the more concerning side, deepfake videos — where a real person's face is convincingly replaced with someone else's — are powered by GAN technology. On the creative side, game studios and film studios use GANs to generate realistic character faces, textures, and environments, saving months of manual design work.

Summary

LangChain is a framework for building LLM-powered applications. LLMs are large neural networks that understand and generate human language. RAG is an architecture that grounds LLM responses in real, retrieved documents. FAISS is a library for fast vector similarity search. A Vector is a numerical representation that encodes semantic meaning. A Vector DB is a database purpose-built for storing and querying those vectors. Generative AI is the broader category of AI systems that create new original content. GANs are a specific generative architecture where two networks compete to produce increasingly realistic synthetic data.